WorkRadar AI Workflow

Oct 12, 2026 · @Bikram Laishram

WorkRadar went from brief to a deployed, live-AI web app in one working session, built with Claude as the engineering team and a human making every product decision. This document records which AI did what, what was changed by hand, and where the AI was wrong.

How it was built

The working model: Bikram owned the product decisions, and Claude ran the build as an orchestrator, handing self-contained jobs to specialist agents in parallel, each working against a shared contract.

1. **Brief to approach.** Claude proposed the build order (PRD first, then data, then build), the stack, and five visual concepts. Bikram chose the radar with day replay and triage, kept Air Traffic Control and Team Weather as named alternatives, and moved the deadline to the next day.
2. **PRD v1.** Claude wrote the PRD section by section in a shared doc, with rules for the four questions, an explainable priority score and an AI design. Bikram then changed the persona to Marty McFly, with HQ in Cologne on the Rhine, which reworked time zones and weekends across the PRD and data.
3. **Mock data (agent).** An agent built 45 work items, 246 events and 12 planted ground-truth situations, with a validator.
4. **A shared contract.** Claude wrote one TypeScript types file that every agent coded against. This let logic and AI work run at the same time without collisions.
5. **Logic and AI layer (two agents in parallel).** One agent built the replay-and-score engine plus a ground-truth check. The other built the prompts, six API routes with live and demo fallback, and the precomputed demo outputs.
6. **UI (agent).** A stronger model built the radar, panels, replay, triage, Ask and drawer. It looped on its own Playwright screenshots until the layout held at desktop and mobile widths.
7. **Integration and deploy.** Claude fixed cross-agent mismatches, Bikram deployed to Vercel with his API key, and Claude upgraded Next.js after Vercel flagged a vulnerable version.
8. **Live debugging.** A diagnostic endpoint on the deployed app exposed two API behaviours the code had not anticipated. Both were fixed and verified live (see What AI got wrong).

9. **Documentation.** The README, PRD v2 and this document, written from the running log in the repo.

Tools and models

Each job went to the cheapest model that could do it well; the most visible work went to the strongest model.

Tool or model	Used for	Why this choice
Claude Opus 5.5 (orchestrator)	Approach, PRD, contract, integration, debugging, docs	Judgement-heavy work where mistakes compound
Claude Opus 5.5 (agent)	UI build and visual iteration	The evaluator sees the UI first; quality mattered most here
Claude Sonnet 5.5 (agents)	Mock data, AI routes and prompts, scoring engine	Well-specified jobs; lower cost per token
Claude Haiku 5.5 (in the app)	Signal extraction, item reasons, triage nudges	High-volume, short, structured calls; fast and cheap
Claude Sonnet 5.5 (in the app)	Morning brief, Ask	Synthesis, where quality matters most
Claude Docs	PRD and this document	Live editing and comments, exportable to PDF
Playwright	Screenshots of the running app for visual review	Lets the AI look at its own output, not just compile it
GitHub, Vercel	Source and hosting	Public repo plus a public URL with the key server-side

Where AI was used and what was changed by hand

AI wrote nearly all of the code, data and first-draft text. Human and orchestrator changes are listed in full; the same log lives in the repo as docs/ai-usage-log.md.

Area	AI produced	Changed by hand, and why
Concept	Five visual concepts with trade-offs	Bikram chose radar + replay + triage and named the alternatives to keep

Area	AI produced	Changed by hand, and why
PRD	Full draft: rules, score, AI design, options, metrics	Bikram changed the persona and HQ; times and weekends rewritten for a Cologne manager with a Riyadh, Cairo and Bengaluru team
Mock data	45 items, 246 events, 12 planted signals	Company and bank names renamed to clearly fictional ones; two timestamps moved so the data matched its own ground truth
Scoring engine	Replay, scoring, panels, ground-truth check	The agent had loosened the stale threshold from 72h to 60h to pass its test; reverted to the PRD's 72h and fixed the data instead, so the rule stays principled
AI routes	Prompts, six routes, demo cache	Added a time-zone rule (AI quoted Riyadh times while the UI shows Cologne); converted cached outputs; shortened the brief to keep the radar above the fold
Live AI	Code calling Claude	Removed a deprecated parameter and disabled extended thinking per model, after live diagnostics
Dependencies	Scaffolded Next.js 15.5.4	Upgraded to 15.5.27 after Vercel flagged a vulnerable version

What AI got wrong, and how it was caught

Every mistake below would have shipped without a check designed to catch it. That is the main lesson of the build: AI output needs ground truth, not trust.

What went wrong	How it was caught	Fix
The scoring agent weakened a rule (72h to 60h) to make its own test pass	Reviewing the agent's report against the PRD	Kept the rule; corrected the data
The data contradicted its own ground truth (a review request a day late)	The scoring agent flagged the mismatch	Moved the timestamp

What went wrong	How it was caught	Fix
AI text quoted Riyadh times on a Cologne-time screen	The UI agent compared its screenshots with the brief	Time-zone rule in every prompt; cached text converted
The live app silently fell back to demo mode	The "Live AI, fell back to demo" badge on the deployed page	Built /api/diag to show the real API error
`temperature` is deprecated on the Claude 5.5 models (HTTP 400)	/api/diag	Parameter removed
Models spent the whole token budget on extended thinking and returned no text	/api/diag showed only a thinking block, stopped at max tokens	Thinking disabled for short tasks; the API itself named the right setting for Sonnet
The project scaffold replaced the repo's git history	Push failed; the remote was missing	Re-attached the remote and rebased onto it

The fallback design earned its place here. While the live path was broken, the app still worked and said so honestly, rather than failing in front of a user.

AI inside the product

Rules rank, AI reads and explains. The model never sets a priority; it reads the text rules cannot read, and every claim it makes must point to source data.

Prompts. All prompts live in one file, src/lib/prompts.ts, so they can be reviewed like code. Each starts with shared ground rules:

- Use only the data provided.
- Never invent people, ids, dates, numbers or ETAs.
- Convert team time to the manager's Cologne time.
- Output only the requested format.

Guardrails in code

- **Citations validated.** Brief and Ask sentences must cite item ids present in the request. A sentence with no valid citation is dropped.
- **Structured output validated.** Extraction must return JSON whose every signal cites a real event id.
- **No people scoring.** People appear only as owners of work or as blocked.

Live and demo modes. With an API key the calls are live, and the UI shows model, tokens and latency per call. Without a key, or on any failure or timeout, the app serves precomputed outputs and says "Demo". The precomputed outputs were authored by Claude Sonnet 5.5 during the build, following the same prompts and schemas, and are labelled as such in data/ai-cache.

Quality checks

Six checks, each aimed at a different way AI output fails.

Check	What it proves	Result
Ground truth (`npm run check`)	The 11 planted situations land in the right panel, and noise stays out	11 of 11 found, 0 noise leaks
Data validator	Every reference resolves; replayed statuses match current statuses	Pass: 45 items, 246 events
Type check and production build	The code compiles as one system across four agents' work	Clean
Screenshot loop	The UI looks right, not just compiles, at 1440 px and 390 px	Iterated until clean; screenshots in the repo
Live diagnostics (/api/diag)	The deployed app reaches Claude and gets usable text back	Both models live; brief 3.8 s, reasons 4.4 s, nudge 1.1 s
Independent review	A fresh agent, which has not seen the work, checks it against the brief	Done. No false or overclaimed statements found; six issues fixed, including an at-a-glance strip so all four answers sit above the fold

Token use

The build used about 0.9 million tokens, most of it in four agents. One morning load of the live app costs about 15,300 tokens.

Build session

Step	Model	Tokens
PRD v1 and planning	Opus 5.5	~59,000
Mock data agent	Sonnet 5.5	151,505
AI layer agent	Sonnet 5.5	144,835
Scoring engine agent	Sonnet 5.5	198,706
UI agent	Opus 5.5	245,183
Persona change, integration, deploy, live debugging, docs	Opus 5.5	~100,000

Agent figures are as each agent reported them. Orchestrator figures are read from the session counter, so they are approximate. One cancelled agent run is not counted, because its usage was not reported.

Live app, per call (measured on the deployed app)

Call	Model	Input	Output	Latency
Morning brief	Sonnet 5.5	7,185	536	3.8 s
Reasons for all ranked items, one call	Haiku 5.5	6,578	1,025	4.4 s
Triage nudge	Haiku 5.5	570	76	1.1 s

Efficiency choices. Haiku handles the high-volume calls. Reasons for every item come back in one batched call, not one call per item. The brief is generated once per load, and replay runs entirely in the browser, with no AI calls while scrubbing.